Extremely Randomised Trees

2026-04-17

Introduction

Extremely Randomised Trees — usually shortened to "Extra Trees" — are a variant of random forests introduced by Geurts, Ernst, and Wehenkel in 2006. The defining innovation: instead of optimising the split threshold at each node, Extra Trees draws the threshold *at random* from the feature's range. The additional source of randomness increases bias slightly but reduces variance further, often producing models with accuracy competitive to random forests at a fraction of the training time.

Prerequisites

A working understanding of random forests, decision-tree splitting criteria (Gini, variance), and the bias-variance trade-off in ensemble methods.

Theory

At each node, Extra Trees:

1. Choose m random features (as in random forest).
2. For each chosen feature, draw a *random threshold* uniformly from its observed range — not the optimal split.
3. Split on whichever (feature, random-threshold) pair gives the lowest impurity among these candidates.

Trees are typically fit to the entire training data without bootstrap sampling (`replace = FALSE`); variance reduction comes from the random thresholds instead of the bootstrap. The combination of feature subsampling and threshold randomisation produces highly decorrelated trees, and averaging over many of them produces a low-variance ensemble.

Assumptions

Feature importance is approximately diffuse — random thresholds work poorly when a few dominant informative features need precise split points; in those settings, optimising thresholds (random forest) outperforms.

R Implementation

```
library(ranger)

set.seed(2026)
n <- 500
df <- data.frame(
  x1 = rnorm(n), x2 = rnorm(n), x3 = rnorm(n), x4 = rnorm(n),
  y  = sin(2 * rnorm(n)) + 0.3 * rnorm(n)
)

# ranger implements extra-trees via `splitrule = "extratrees"`
rf <- ranger(y ~ ., data = df,
```

```r
              num.trees = 500, mtry = 2,
              splitrule = "variance")

et <- ranger(y ~ ., data = df,
              num.trees = 500, mtry = 2,
              splitrule = "extratrees",
              num.random.splits = 5)

c(rf_oob_rmse = sqrt(rf$prediction.error),
  et_oob_rmse = sqrt(et$prediction.error))
```

Output & Results

OOB-based RMSE for both random forest and Extra Trees on the same data. Extra Trees typically produces slightly lower RMSE on noisy datasets where variance reduction dominates; random forest typically wins on structured signals where threshold optimisation captures sharper boundaries.

Interpretation

A reporting sentence: "Extra Trees with `num.random.splits = 5` produced marginally lower OOB RMSE (0.38) than random forest (0.41) on this noisy regression task; the variance reduction from random thresholds outweighed the small bias cost. Training was approximately 3 × faster than random forest." Always state the number of random splits and compare against random forest baseline.

Practical Tips

- Extra Trees can skip bootstrapping (`replace = FALSE`) for tighter variance; random forest keeps the bootstrap as its primary variance-reduction mechanism.
- With many informative features, tune `num.random.splits` upward (10–20) so the random-threshold candidates include the optimum more often; this approaches random forest behaviour.
- Training is typically 2–5× faster than random forest for the same tree count, because no threshold-optimisation search is performed at each split.
- Use random forest when a few features dominate the signal; use Extra Trees when signal is diffuse across many features.
- For variable importance, use permutation importance (`importance = "permutation"`); impurity-based importance is biased in Extra Trees just as in random forests.
- For very large datasets, Extra Trees' speed advantage compounds; consider it as the default ensemble when training time matters.

R Packages Used

`ranger` with `splitrule = "extratrees"` for the canonical implementation; `extraTrees` package for the Python-compatible reference implementation; `parsnip::rand_forest()` (with engine selection) for tidymodels integration; `caret` for cross-validated tuning of `num.random.splits`.

For Reviewers

What to look for in a paper using this method.

- **Common misapplications.**
- **Diagnostics that should be reported but often aren't.**
- **Red flags in tables and figures.**
- **What to verify.**
- **What an adequate Methods paragraph must contain.**

See also — labs in this chapter

- Cross-validation, nested CV, bootstrap .632+
- Regularisation: ridge, lasso, elastic net
- Trees, random forests, gradient boosting
- Interpretability and SHAP
- Tabular neural networks with torch
- Imaging and sequence models intro
- tidymodels pipelines
- FDR, knockoffs, replication crisis